

Arquitectura completa para un configurador 3D de cocinas tipo Leroy Merlin

Objetivo del proyecto

Crear una aplicación web que permita:

- Diseñar cocinas en 3D.
- Arrastrar módulos y muebles.
- Configurar materiales y colores.
- Calcular precios automáticamente.
- Guardar proyectos.
- Gestionar módulos desde un panel de administración.
- Escalar el sistema profesionalmente.

La idea es construir un sistema similar a los configuradores de:

- Leroy Merlin
- IKEA Planner
- Schmidt
- Ixina

Arquitectura general

Frontend Web (React + Three.js)



Backend API (NestJS)



MariaDB



Storage modelos 3D (.glb)

Tecnologías recomendadas

Frontend

Recomendado

- Next.js
- React
- React Three Fiber
- Drei
- Zustand
- React Use Gesture

Librerías principales

Three.js

Motor gráfico 3D para navegador.

React Three Fiber

Adaptación moderna de Three.js para React.

Permite:

- Renderizado 3D
- Cámaras
- Luces
- Modelos GLB
- Raycasting
- Colisiones
- Interacción drag & drop

Drei

Helpers para React Three Fiber.

Muy útil para:

- OrbitControls
- Grid
- Gizmos
- Carga de modelos

Zustand

Gestión de estado global.

Perfecto para:

- Objetos de escena
 - Selección
 - Historial undo/redo
 - Configuración de cocina
-

Backend

NestJS

Framework recomendado.

Ventajas:

- APIs limpias
 - Escalable
 - Modular
 - Auth integrada
 - WebSockets
 - Compatible con MariaDB
-

Base de datos

MariaDB

La base de datos ya existente puede utilizarse perfectamente.

Ventajas:

- Muy rápida
 - Compatible con MySQL
 - Fácil integración con Prisma
 - Ideal para catálogos y proyectos
-

ORM recomendado

Prisma

Permite trabajar con MariaDB fácilmente.

Ventajas:

- Tipado automático
- Queries limpias
- Migraciones
- Muy rápido de desarrollar

Sistema de almacenamiento

Modelos 3D

Los archivos 3D NO deben guardarse en MariaDB.

Se recomienda:

- AWS S3
- Cloudinary
- Carpeta uploads
- CDN

En la base de datos solo se guarda la URL.

Ejemplo:

```
{  
  "model_url": "/uploads/models/mueble80.glb"  
}
```

Formato 3D recomendado

GLB / GLTF

Formato estándar para web moderna.

Ventajas:

- Muy rápido
- Compatible con Three.js
- Soporta texturas
- Ligero

- Optimizado

Herramientas recomendadas:

- Blender
- SketchUp
- 3ds Max

Estructura del sistema

1. Catálogo de módulos

Cada módulo debe contener:

- Medidas
- Precio
- Modelo 3D
- Materiales
- Categoría
- Reglas
- Snap points

Ejemplo:

```
{
```

```
  "id": 12,
```

```
  "name": "Mueble bajo 80",
```

```
  "category": "base_unit",
```

```
  "width": 80,
```

```
  "height": 90,
```

```
  "depth": 60,
```

```
  "price": 250,
```

```
  "model_url": "/models/base80.glb",
```

```
  "anchors": {
```

```
"left": true,
```

```
"right": true
```

```
}
```

```
}
```

Sistema SNAP

Qué es

El sistema snap es el encargado de unir automáticamente los muebles.

Ejemplo:

- Un módulo se acerca a otro.
- El sistema detecta distancia.
- Se alinean automáticamente.

Esto es lo que hace que parezca profesional.

Cómo funciona el SNAP

Proceso

1. Usuario mueve módulo.
2. Se detectan módulos cercanos.
3. Se calculan puntos de anclaje.
4. Se ajusta posición automáticamente.

Ejemplo:

```
if(distance < 0.2){
```

```
  snapTo(module.right)
```

```
}
```

Grid invisible

Se recomienda trabajar con:

- Grid de 10 cm
- O medidas reales

Ventajas:

- Todo encaja

- Evita huecos
- Facilita restricciones

Sistema de colisiones

Debe impedir:

- Muebles superpuestos
- Objetos atravesándose
- Colocaciones inválidas

Se utiliza:

- BoundingBox
- Raycasting
- Physics Engine opcional

Motor de restricciones

Sistema encargado de validar reglas.

Ejemplos:

- Un horno solo puede ir en ciertos muebles.
- Un módulo alto necesita pared.
- Un fregadero necesita encimera.

Guardado de proyectos

La cocina NO se guarda como imagen.

Se guarda como JSON.

Ejemplo:

```
{  
  "room": {  
    "width": 420,  
    "height": 250  
  },
```

```
  "modules": [
```

```
{  
  "module_id": 12,  
  "x": 0,  
  "z": 0,  
  "rotation": 0  
}  
]  
}
```

Estructura SQL recomendada

Tabla módulos

```
CREATE TABLE kitchen_modules (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  name VARCHAR(255),  
  category VARCHAR(100),  
  
  width FLOAT,  
  height FLOAT,  
  depth FLOAT,  
  
  price DECIMAL(10,2),  
  
  model_url TEXT,  
  thumbnail_url TEXT,  
  
  snap_points JSON,  
  rules_json JSON,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla proyectos

```
CREATE TABLE kitchen_projects (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  user_id INT,  
  name VARCHAR(255),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla elementos colocados

```
CREATE TABLE kitchen_project_items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  project_id INT,  
  module_id INT,  
  
  pos_x FLOAT,  
  pos_y FLOAT,  
  pos_z FLOAT,  
  
  rot_y FLOAT,  
  
  FOREIGN KEY (project_id)  
    REFERENCES kitchen_projects(id),  
  
  FOREIGN KEY (module_id)  
    REFERENCES kitchen_modules(id)  
);
```

Panel de administración

Objetivo

Permitir al administrador:

- Añadir módulos
 - Editar precios
 - Subir modelos GLB
 - Gestionar categorías
 - Gestionar materiales
 - Gestionar colores
-

Solución recomendada

Strapi

Ventajas:

- Conecta con MariaDB
 - CRUD automático
 - Login admin
 - Upload de archivos
 - API REST
 - API GraphQL
-

Flujo completo del sistema

ADMIN



Sube modelo GLB



Strapi guarda datos en MariaDB



Frontend obtiene catálogo vía API



Usuario arrastra módulos



Sistema aplica snap



Se guarda proyecto



La cocina puede reabrirse después

Sistema Drag & Drop

Funcionamiento

1. Usuario arrastra módulo.
 2. Se crea instancia 3D.
 3. Se mueve sobre la escena.
 4. Se calcula colisión.
 5. Se aplica snap.
 6. Se guarda posición.
-

Sistema Undo / Redo

Muy recomendable.

Permite:

- Deshacer movimientos
- Recuperar acciones
- Mejor UX

Se puede implementar con Zustand.

Sistema de materiales

Cada módulo puede tener:

- Colores
- Texturas
- Materiales
- Acabados

Ejemplo:



```
"material": "roble",  
"texture": "/textures/oak.jpg"  
}
```

Sistema de precios

Cada módulo tiene precio.

El sistema calcula automáticamente:

- Total cocina
- IVA
- Extras
- Electrodomésticos

Ejemplo:

```
const total = modules.reduce((a,m)=>a+m.price,0)
```

Funciones profesionales recomendadas

Vista 2D + 3D

Muy útil.

Permite:

- Plano técnico
 - Medidas
 - Distribución rápida
-

Exportación PDF

Muy recomendable.

Debe incluir:

- Imagen cocina
 - Lista módulos
 - Precios
 - Medidas
 - Presupuesto
-

Optimización importante

Modelos ligeros

Ideal:

- Menos de 5 MB por módulo
 - Pocos polígonos
 - Texturas comprimidas
-

Seguridad

Recomendaciones

- JWT Auth
 - Roles admin/user
 - Validación uploads
 - Limitar tamaño archivos
-

Escalabilidad futura

El sistema puede crecer hacia:

- IA para diseñar cocinas
 - Realidad aumentada
 - Renderizado realista
 - Exportación CAD
 - Configurador móvil
 - Integración ecommerce
-

Roadmap recomendado

FASE 1 — MVP

Objetivo

Crear sistema básico funcional.

Incluye

- Habitación simple
- Añadir módulos
- Mover módulos

- Rotar módulos
- Guardar proyecto
- Login básico

FASE 2 — Sistema profesional

Añadir

- Snap automático
- Colisiones
- Grid
- Medidas
- Restricciones

FASE 3 — Experiencia avanzada

Añadir

- Materiales
- Colores
- Presupuestos
- PDF
- Vista 2D

FASE 4 — Nivel Leroy Merlin

Añadir

- IA
- Render realista
- Realidad aumentada
- Optimización avanzada
- Compatibilidad móvil total

Arquitectura específica para tu proyecto actual (Laravel + React + MariaDB)

Stack actual

Tu proyecto ya parte de una base muy buena:

- Laravel

- React

- MariaDB

Por tanto, NO necesitas crear un backend separado con NestJS.

La mejor solución es integrar el configurador 3D como un módulo interno de tu aplicación actual.

Arquitectura recomendada para tu caso

Laravel

- └─ API REST
- └─ Auth
- └─ Panel Admin
- └─ MariaDB
- └─ Storage modelos GLB
- └─ Gestión proyectos

React

- └─ Web principal
- └─ Kitchen Designer 3D
 - └─ Three.js
 - └─ React Three Fiber
 - └─ Drag & Drop
 - └─ Snap System
- └─ Configurador

Qué hará Laravel

Laravel se encargará de:

Backend principal

- Usuarios
- Login
- Roles

- API REST
 - CRUD de módulos
 - Guardado de proyectos
 - Upload de archivos GLB
 - Gestión de materiales
 - Gestión de precios
 - Panel administración
-

Qué hará React

React se encargará únicamente del editor 3D.

Ejemplo:

/kitchen-designer

Dentro de esta sección:

- Catálogo módulos
 - Escena 3D
 - Movimiento objetos
 - Snap
 - Rotaciones
 - Configuración visual
-

Estructura recomendada en Laravel

API endpoints

/api/kitchen/modules

/api/kitchen/projects

/api/kitchen/project/save

/api/kitchen/project/load

/api/kitchen/upload

Estructura recomendada React

resources/js/

└─ kitchen-designer/

- | └─ components/
- | └─ scene/
- | └─ store/
- | └─ hooks/
- | └─ utils/
- | └─ editor.jsx
- | └─ api.js

Librerías recomendadas

Frontend

React Three Fiber

Motor principal 3D.

Drei

Helpers y controles.

Zustand

Estado global.

React Use Gesture

Drag & drop.

Sistema recomendado de almacenamiento

Modelos 3D

Los modelos .glb deben almacenarse:

/storage/app/public/models/

Y Laravel expondrá:

/storage/models/model.glb

Ejemplo subida archivos Laravel

```
$modelPath = $request->file('model')->store('models', 'public');
```

Sistema recomendado para admin

Filament

Se recomienda usar Filament para el panel de administración.

Permite:

- CRUD automático
- Gestión usuarios
- Uploads
- Formularios
- Tablas
- Roles
- Permisos

Estructura SQL adaptada a Laravel

Tabla módulos

```
CREATE TABLE kitchen_modules (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  name VARCHAR(255),
```

```
  category VARCHAR(100),
```

```
  width FLOAT,
```

```
  height FLOAT,
```

```
  depth FLOAT,
```

```
  price DECIMAL(10,2),
```

```
  model_url TEXT,
```

```
  thumbnail_url TEXT,
```

```
  snap_points JSON,
```

```
  rules_json JSON,
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

Tabla proyectos

```
CREATE TABLE kitchen_projects (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  user_id INT,
```

```
  name VARCHAR(255),
```

```
  scene_json LONGTEXT,
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

Guardado recomendado

La forma más eficiente es guardar TODA la escena en JSON.

Ejemplo:

```
{
```

```
  "room": {
```

```
    "width": 400,
```

```
    "height": 250
```

```
  },
```

```
  "items": [
```

```
    {
```

```
      "module_id": 12,
```

```
      "position": [0,0,0],
```

```
      "rotation": [0,1.57,0]
```

```
    }
```

```
  ]
```

```
}
```

Ventajas:

- Mucho más simple
- Fácil de reconstruir
- Compatible con React
- Compatible con Three.js
- Fácil de exportar

Cómo cargar modelos GLB en React

```
import { useGLTF } from '@react-three/drei'
```

```
export default function KitchenModule({ url }) {
```

```
  const { scene } = useGLTF(url)
```

```
  return <primitive object={scene} />
```

```
}
```

Qué será realmente difícil

Lo complejo NO será Laravel.

Las partes complejas serán:

Snap automático

Alineación inteligente entre muebles.

Colisiones

Evitar superposiciones.

UX drag & drop

Que el movimiento sea fluido.

Restricciones

Ejemplo:

- módulos compatibles
- esquinas
- electrodomésticos

- alturas

Roadmap recomendado para tu proyecto

FASE 1

MVP básico

Objetivo

Tener el editor funcionando.

Funciones

- Habitación básica
- Añadir módulos
- Mover módulos
- Rotar módulos
- Guardar JSON
- Cargar proyectos

FASE 2

Sistema profesional

Añadir

- Snap
- Grid
- Colisiones
- Medidas
- Restricciones

FASE 3

Configuración avanzada

Añadir

- Materiales
- Colores
- Texturas
- Presupuestos

- PDF

FASE 4

Nivel comercial avanzado

Añadir

- Render realista
- IA
- Realidad aumentada
- Vista móvil
- Render GPU

Recomendación final para tu caso

Tu stack actual es ideal para este proyecto.

La mejor arquitectura sería:

- Laravel como backend principal
- React como frontend del editor
- React Three Fiber para el 3D
- Filament como panel admin
- MariaDB como base de datos
- GLB como formato estándar

Con esta arquitectura puedes construir un configurador profesional tipo Leroy Merlin completamente escalable.

Conclusión

Para construir un configurador tipo Leroy Merlin no basta con renderizar modelos 3D.

La clave está en:

- Sistema modular
- Reglas inteligentes
- Snap automático
- Restricciones
- UX fluida

- Persistencia de escenas

La arquitectura recomendada actualmente es:

- Next.js
- React Three Fiber
- Three.js
- Zustand
- NestJS
- MariaDB
- Strapi
- Modelos GLB

Este stack permite crear una plataforma moderna, escalable y profesional.